

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1003	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	PRIYADHARSHINI K S	Reg. No.:	192511248
Date:	03/08/2026	Duration:	60 minutes

Code of Conduct

I ____PRIYADHARSHINI K S__192511248_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __PRIYADHARSHINI K S_____

Vehicle Service Scheduling and Predictive Maintenance Management System

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

The core objectives that the proposed system must fulfil are:

- Allow customers to register vehicles and book, reschedule, or cancel service appointments through a web or mobile interface.
 - Continuously capture real-time vehicle telemetry from OBD-II/IoT sensors installed in the vehicle.
 - Apply predictive analytics/machine learning to forecast component wear and the likelihood of failure before it occurs.
 - Proactively notify customers and service centers of upcoming maintenance needs and recommended service windows.
 - Enable service centres to manage technician allocation, spare-parts inventory, and job scheduling efficiently.
 - Maintain a complete, auditable service history for every vehicle to support warranty, resale, and compliance needs.
 - Provide administrators and fleet managers with analytics dashboards for fleet health, service load, and cost trends.
-
- Analyse the functional and non-functional requirements that influence the software architecture.

FUNCTIONAL REQUIREMENTS:

Functional Requirement	Description
User & Vehicle Registration	Customers, technicians, and administrators can register/authenticate; customers can add and manage vehicle profiles.
Service Scheduling	Customers can browse available slots and book, reschedule, or cancel appointments at a chosen service center.
Real-Time Telemetry Ingestion	The system continuously ingests sensor data (engine, oil, battery, brakes, mileage) from OBD-II/IoT devices.
Predictive Maintenance Analysis	ML models analyze telemetry to estimate failure probability and remaining useful life of key components.
Automated Alerts & Notifications	The system sends SMS/email/push alerts when a predicted risk crosses a defined threshold, with suggested service slots.
Service History Management	All inspections, repairs, and part replacements are logged against the vehicle's service record.
Inventory & Parts Management	Tracks spare-parts stock levels across service centers and triggers reorder alerts.
Technician & Resource Allocation	Assigns available technicians and bays to confirmed bookings based on skill and workload.
Payment & Invoicing	Generates invoices and processes payments for completed services.
Reporting & Analytics	Provides dashboards on fleet health, service-center performance, and predictive-model accuracy.

NON - FUNCTIONAL REQUIREMENTS:

Quality Attribute	Requirement
Scalability	Must support a growing number of registered vehicles and concurrent sensor data streams (thousands of vehicles transmitting telemetry every few seconds) without performance degradation.
Performance	Predictive alerts must be generated within a near-real-time window (seconds to a few minutes) of anomalous sensor readings; booking pages must respond within 2 seconds.
Reliability	Telemetry ingestion and alerting must not lose or silently drop safety-critical readings, even during partial system failures.
Availability	The scheduling and notification services must be available 24/7, since vehicle faults and breakdowns can occur at any time.
Security	Vehicle location/telemetry and customer personal/payment data must be encrypted in transit and at rest, with role-based access control.
Maintainability	Individual modules (especially the ML prediction engine) must be upgradable or retrainable independently without redeploying the entire system.
Usability	Booking and alert interfaces must be simple and accessible to non-technical customers on both web and mobile.
Interoperability	Must integrate with OBD-II/IoT hardware protocols, third-party payment gateways, and mapping/location services.
Data Integrity & Auditability	All service records and predictive alerts must be traceable and tamper-evident for warranty and compliance purposes.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

Quality Attributes

Scalability

- Supports increasing numbers of users and vehicles.
- Allows future feature additions without affecting performance.
- Suitable for multi-branch service centres.

Maintainability

- Modular design simplifies software updates.
- Bugs can be fixed without affecting other modules.
- Easy integration of new features.

Reliability

- Accurate data storage and retrieval.
- Consistent performance with minimum failures.
- Regular backup and recovery support.

Availability

- Provides 24×7 access to customers.
- Minimizes service interruption.
- Supports continuous online booking.

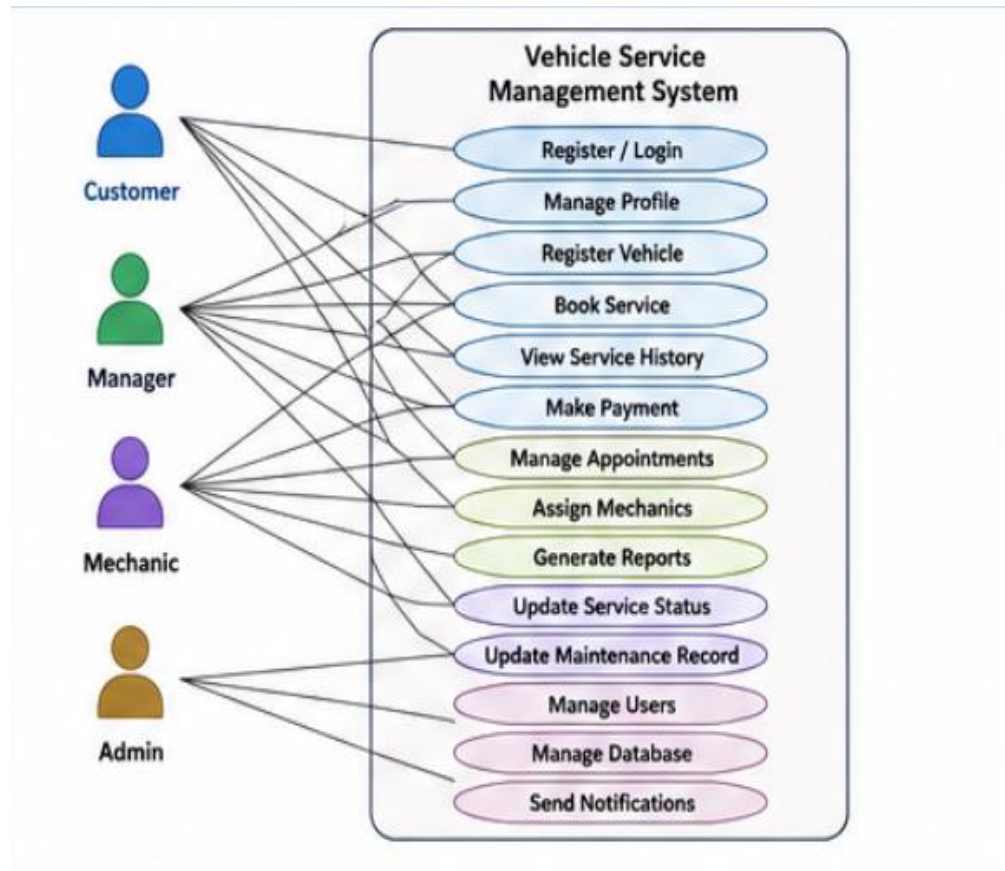
Performance

- Fast processing of service bookings.
- Quick retrieval of maintenance records.
- Efficient handling of multiple user requests.

Security

- Secure login authentication.
- Role-based access control.
- Encrypted password storage.
- Secure database and data protection.

USECASE DIAGRAM:



2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

Selected Software Architecture:

The proposed Vehicle Service Scheduling and Predictive Maintenance Management System adopts a Layered Architecture with the Model-View-Controller (MVC) architectural pattern. This architecture separates the application into different layers, namely the Presentation Layer, Business

Logic Layer, Data Access Layer, and Database Layer. Each layer performs a specific responsibility, making the system more organized, scalable, and maintainable. The MVC pattern further separates the user interface, application logic, and data management, reducing complexity and improving code reusability. This architecture also simplifies testing, debugging, and future enhancements while ensuring efficient communication between system components.

Architecture Layers

1. Presentation Layer (User Interface)

- Customer Portal
- Mechanic Dashboard
- Manager Dashboard
- Administrator Dashboard
- Responsive Web Interface

2. Business Logic Layer

- User Authentication Module
- Vehicle Management Module
- Service Scheduling Module
- Predictive Maintenance Module
- Notification Management Module
- Report Generation Module

3. Data Access Layer

- Database Connectivity
- Data Validation
- CRUD Operations
- Query Processing

4. Database Layer

- User Database
- Vehicle Database

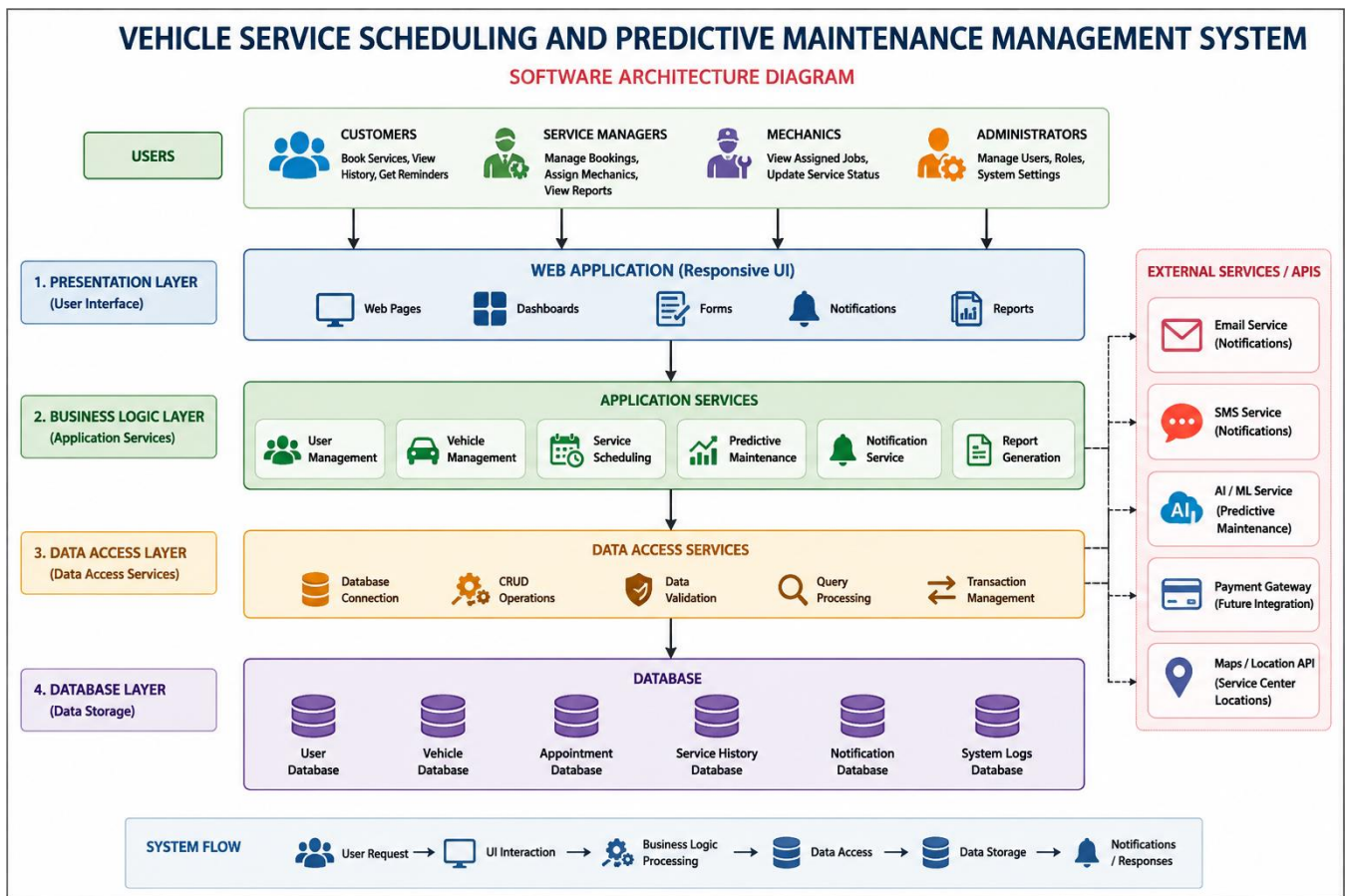
- Service History Database
 - Appointment Database
 - Notification Database
- Justify why the selected architecture is the most suitable for the given problem.

Justification:

The **Layered MVC Architecture** is the most suitable choice because it provides a clear separation of responsibilities between the user interface, business logic, and database operations. This separation improves code organization, making the system easier to develop, test, and maintain. Since the project contains multiple modules such as vehicle registration, appointment scheduling, maintenance history, reminders, and report generation, the layered approach ensures that each module functions independently without affecting others. It also supports scalability, allowing new features such as mobile applications, AI-based predictive maintenance, and cloud integration to be added in the future. Furthermore, MVC improves software reusability, simplifies debugging, and enhances overall system performance, making it an ideal architecture for the proposed system.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.



- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

Major Layers / Components

- Presentation Layer (User Interface)
- Business Logic Layer
- Data Access Layer
- Database Layer

Databases

- User Database
- Vehicle Database
- Service History Database

- Appointment Database
- Notification Database

External Systems

- Email Notification Service
- SMS Notification Service
- Payment Gateway (Future)
- Cloud Storage (Future)

APIs

- REST API
- Email API
- SMS API
- Google Maps API (Future)

Communication Mechanisms

- HTTP/HTTPS Protocol
- REST API Requests
- SQL Database Queries
- JSON Data Exchange
- Secure User Authentication

- Use appropriate software architecture notations and labels.

Use Appropriate Software Architecture Notations and Labels

- User / Actor
- Presentation Layer (User Interface)
- Business Logic Layer (Application Layer)
- Data Access Layer

- Database Layer
- External Systems / Third-Party Services
- REST API
- HTTP / HTTPS Communication
- SQL Database Connection
- Data Flow Arrows (→)
- CRUD Operations
- Authentication & Authorization Module
- Notification Service
- Report Generation Module
- MySQL Database

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

Customer Portal

Purpose: Provides an interface for customers to access the system.

Responsibilities:

- Register and log in.
- Book service appointments.
- View service history.
- Receive maintenance reminders.

Manager Dashboard

Purpose: Helps the service centre manager monitor and manage operations.

Responsibilities:

- Manage service bookings.
- Assign mechanics.
- Monitor service progress.
- Generate reports.

Mechanic Dashboard

Purpose: Supports mechanics in performing vehicle servicing.

Responsibilities:

- View assigned vehicles.
- Update service status.
- Record maintenance details.
- Mark service completion.

Admin Dashboard

Purpose: Controls and manages the overall system.

Responsibilities:

- Manage users.
- Monitor system activities.
- Maintain database.
- Generate system reports.

Business Logic Module

Purpose: Processes user requests and executes business rules.

Responsibilities:

- Validate user requests.
- Schedule vehicle services.
- Manage maintenance records.
- Generate reminders and notifications.

Database

Purpose: Stores all application data securely.

Responsibilities:

- Store customer details.
- Maintain vehicle records.
- Save appointment details.
- Store maintenance history.

External APIs

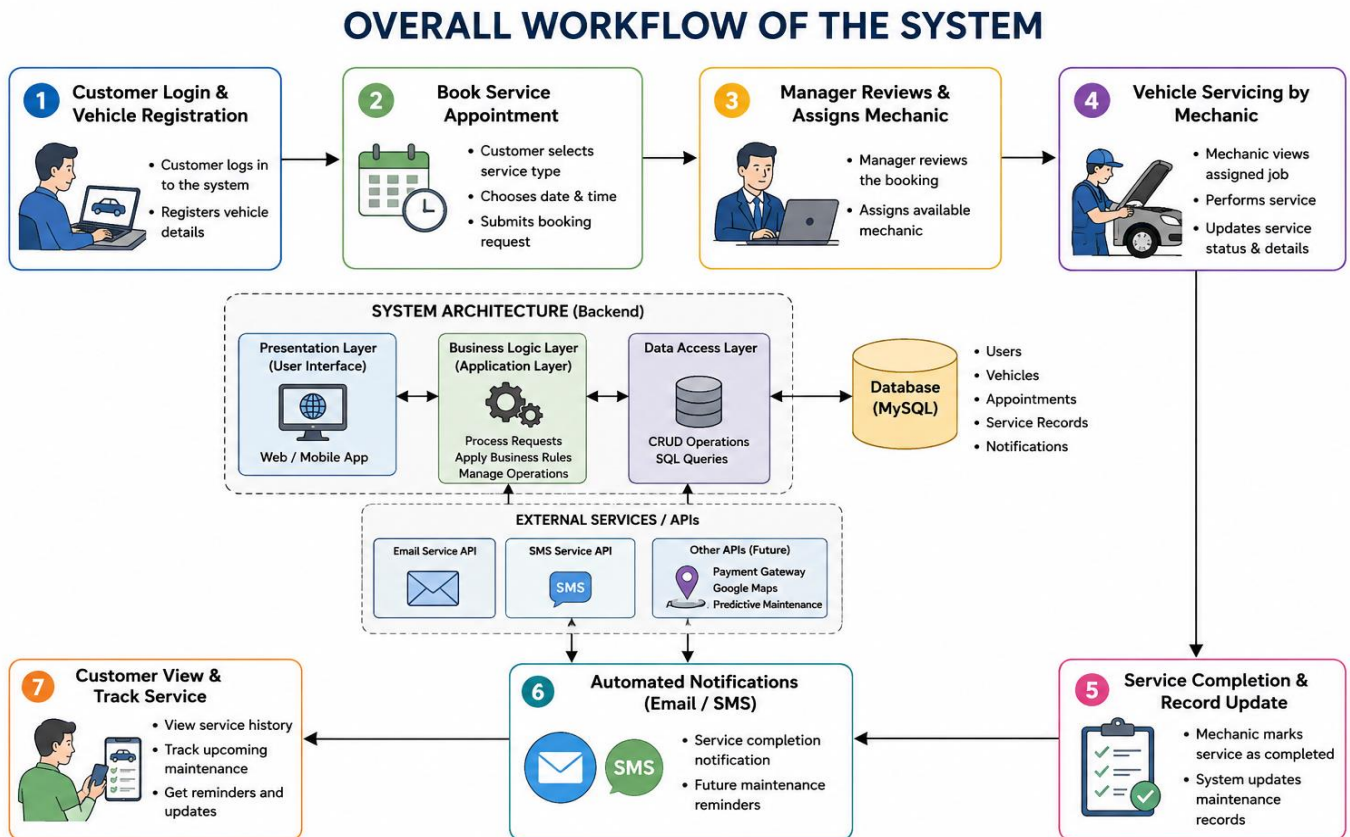
Purpose: Provides additional services through third-party integration.

Responsibilities:

- Send Email notifications.
 - Send SMS reminders.
 - Support payment gateway integration.
 - Connect with Google Maps (future enhancement).
-
- Explain how the components communicate and exchange data.

The components communicate through a layered architecture to ensure efficient data exchange. Users send requests through the Presentation Layer, which forwards them to the Business Logic Layer for processing. The Business Logic Layer validates the requests and communicates with the Data Access Layer to retrieve or update data. The Data Access Layer interacts with the Database Layer using SQL queries. External APIs such as Email and SMS services are used to send notifications and reminders. The processed information is then returned to the Presentation Layer and displayed to the user through HTTP/HTTPS communication.

- Illustrate the overall workflow of the system.



5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- **Scalability**

The Layered MVC Architecture supports scalability by separating the application into independent layers. New modules such as AI-based maintenance, cloud storage, or mobile applications can be added without affecting the existing system.

- **Security**

The architecture improves security through user authentication, role-based access control, encrypted passwords, and secure HTTPS communication. These mechanisms protect customer data and prevent unauthorized access.

- **Performance**

The architecture enhances performance by separating the user interface, business logic, and database operations. This reduces processing time and enables faster service booking, record retrieval, and report generation.

- **Reliability**

The layered design ensures reliable system operation by maintaining accurate data processing and secure database transactions. Regular backups and proper error handling improve system stability and reduce the risk of failures.

- **Maintainability**

The modular structure allows developers to modify or update individual layers without affecting other components. This simplifies software maintenance, bug fixing, and future enhancements.

- **Availability**

The architecture provides continuous system availability through a centralized database and reliable server support. Users can access the application anytime to book appointments, view service history, and receive notifications.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

The Layered MVC Architecture was selected because it provides a clear separation between the user interface, business logic, and database operations. This separation makes the system easier to

develop, test, and maintain while reducing code complexity. It supports modular development, allowing new features to be added without affecting existing components. The architecture also improves security by controlling access through different layers and enhances scalability to support future system expansion. Its organized structure ensures better performance, reliability, and maintainability, making it an ideal choice for the Vehicle Service Scheduling and Predictive Maintenance Management System.

- Discuss the trade-offs considered while selecting the architecture.

While selecting the Layered MVC Architecture, several trade-offs were considered. Although this architecture requires more development time and a well-defined structure, it provides better organization and modularity than a simple monolithic design. Communication between multiple layers may introduce a small performance overhead, but it improves maintainability and scalability. The architecture also requires developers to understand the MVC pattern, increasing the initial learning effort. However, these trade-offs are acceptable because the architecture offers greater flexibility, security, and easier future enhancements for the Vehicle Service Scheduling and Predictive Maintenance Management System.

- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

The Layered MVC Architecture supports future enhancements by allowing new modules, such as AI-based predictive maintenance, mobile applications, and online payment systems, to be added without modifying the existing components. Its modular design simplifies system integration with external services like Email APIs, SMS APIs, Google Maps, and cloud platforms. The clear separation of responsibilities between layers makes debugging, testing, and software updates easier. Developers can maintain or upgrade individual modules without affecting the entire system. This architecture ensures long-term maintainability, flexibility, and efficient support for future technological advancements.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality	Clearly explains component responsibilities,	Explains most components and	Provides limited explanation of interactions and	Component interactions and quality attributes

Attribute Analysis (15%)	interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	quality attributes with minor gaps.	addresses only some quality attributes.	are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25